Генеративные чат-боты

Елена Кантонистова

Как модели учатся понимать язык?

Обучение на текстах: задача 1

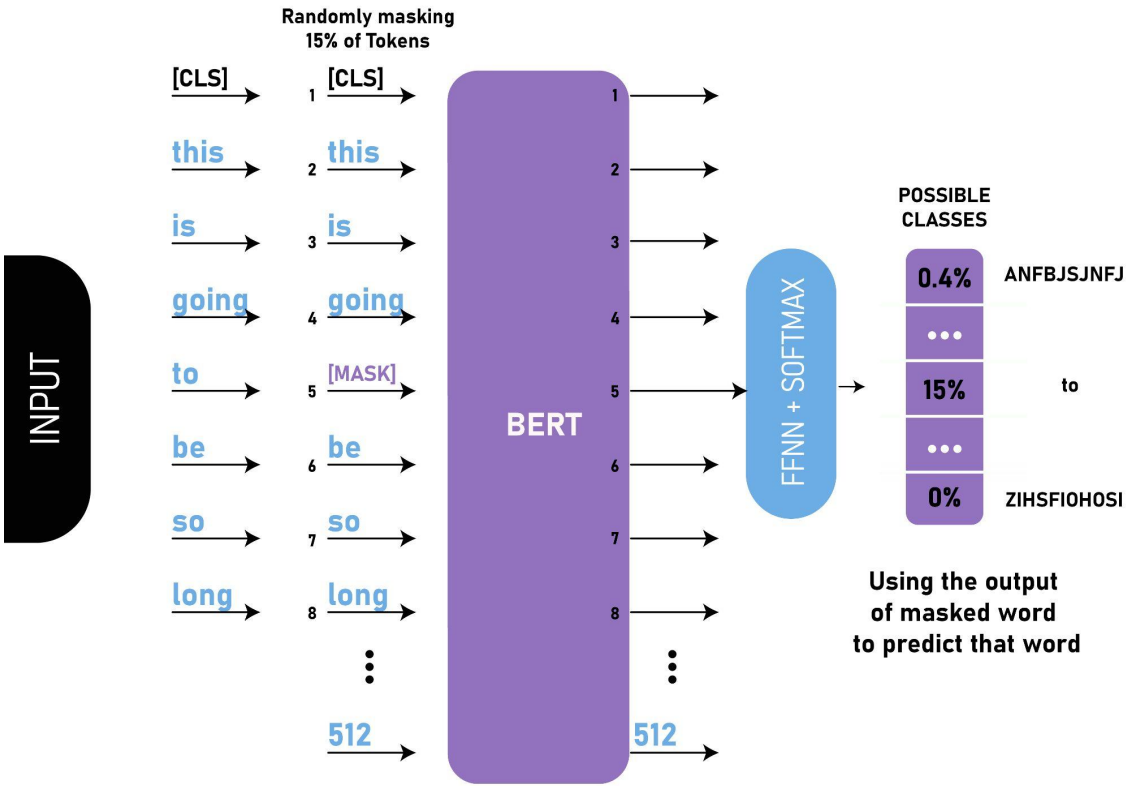
Для обучения используется простой прием: часть слов мас[redacted]ется. Задача нейросети предположить, какие слова б[redacted]ропущены. При этом нейросеть обучается на всех доступных текстах. Это огромнейший[redacted]ив данных.

Обучение на текстах: задача 1

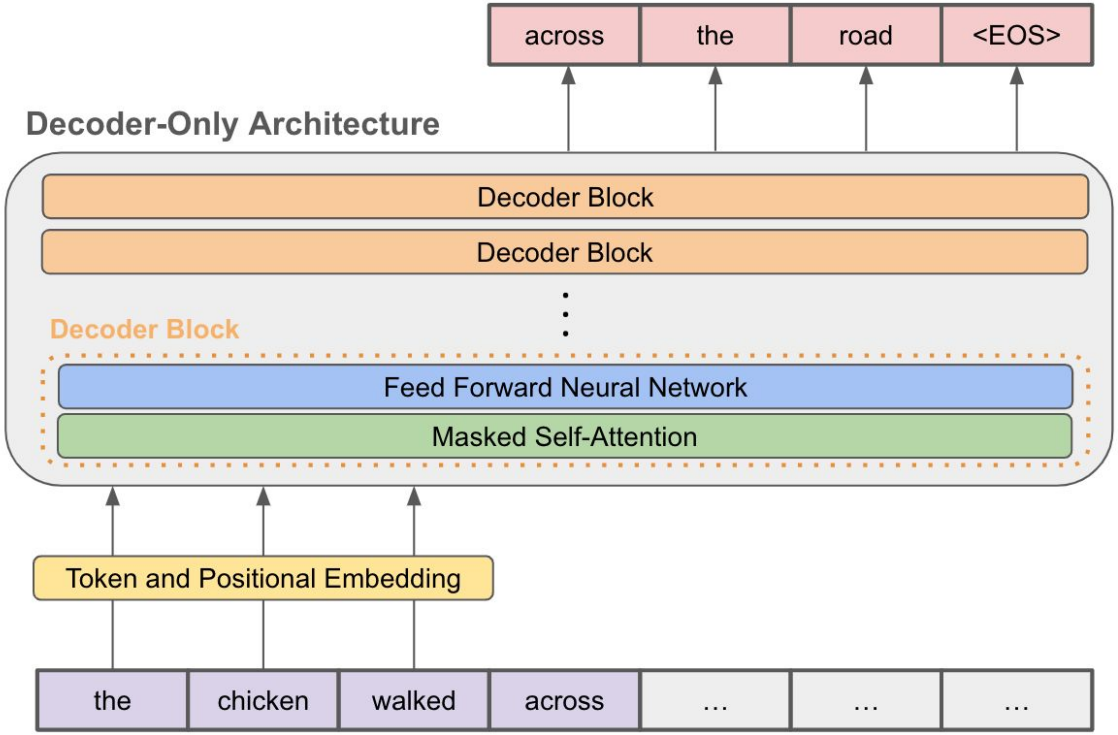
Для обучения используется простой прием: часть слов маскируется. Задача нейросети предположить, какие слова были пропущены. При этом нейросеть обучается на всех доступных текстах. Это огромнейший массив данных.

Обучение на текстах: задача 2

BERT (encoder-модель)

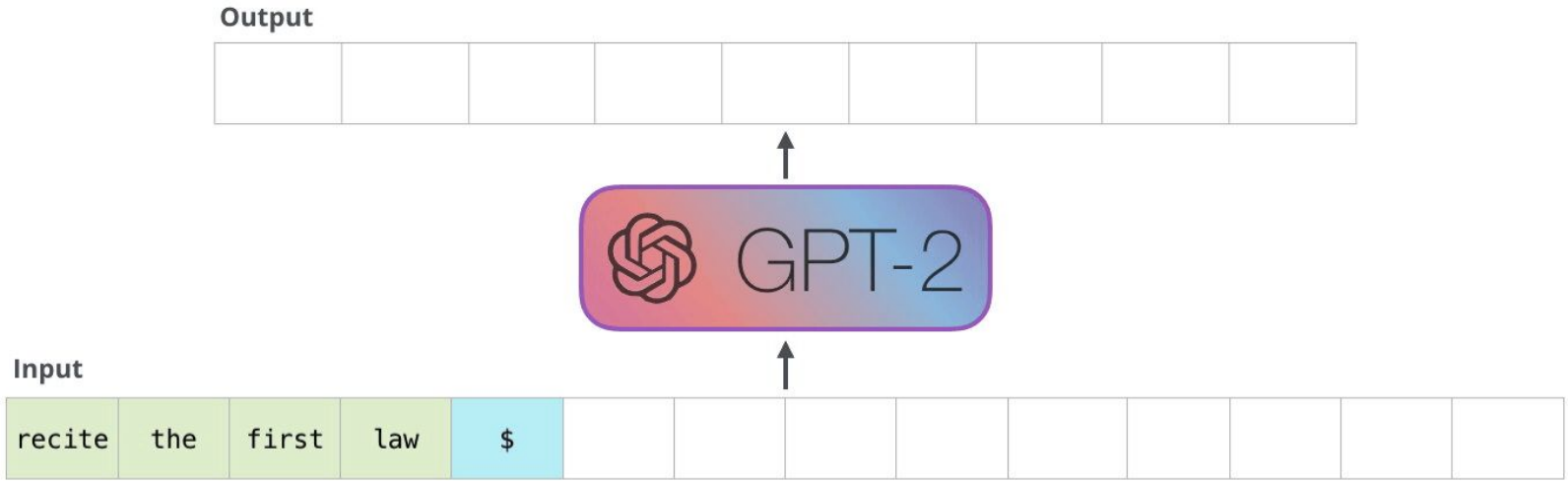


GPT (decoder-модель)

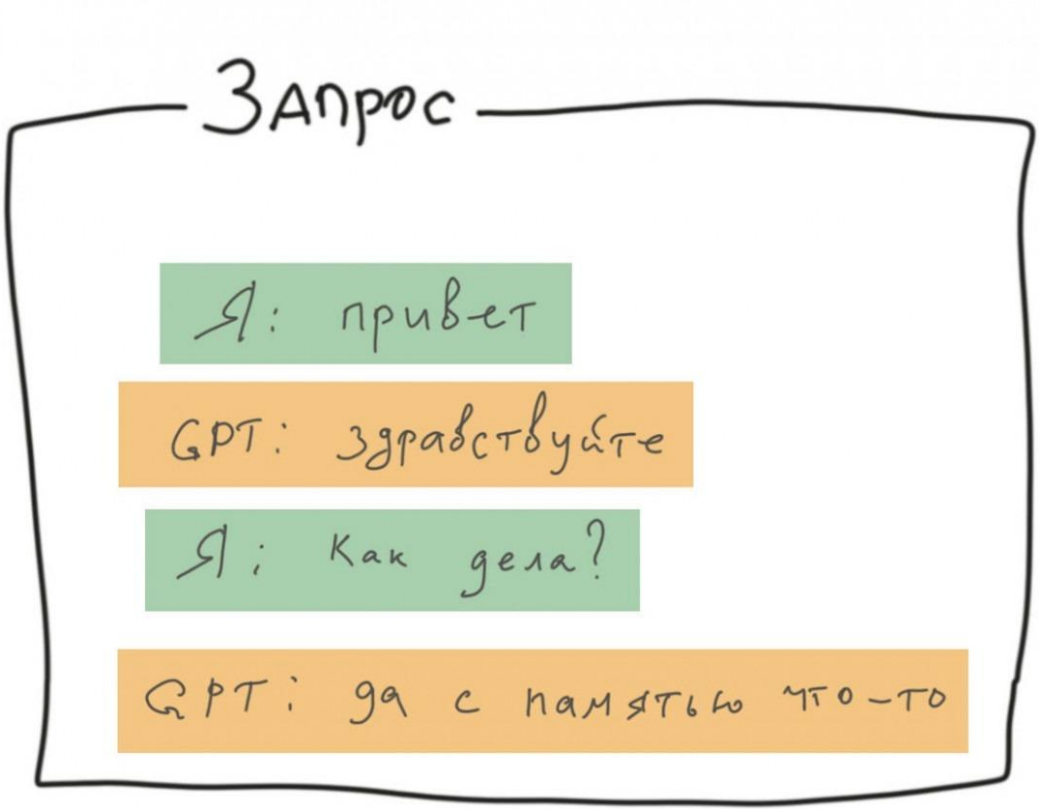


Chat**GPT**

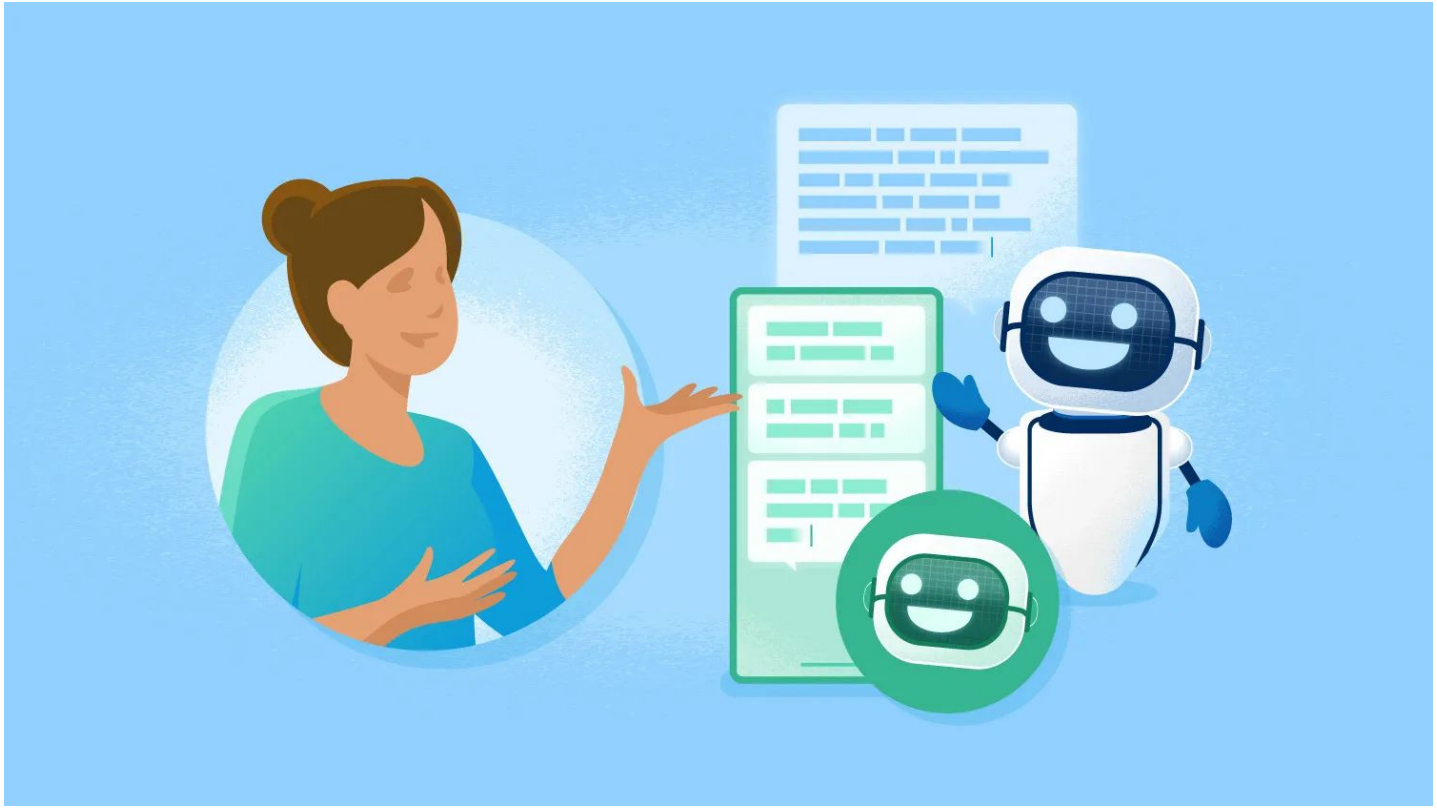
GPT - это генеративная нейронная сеть, которая умеет генерировать продолжение текста по заданному началу (промпту).



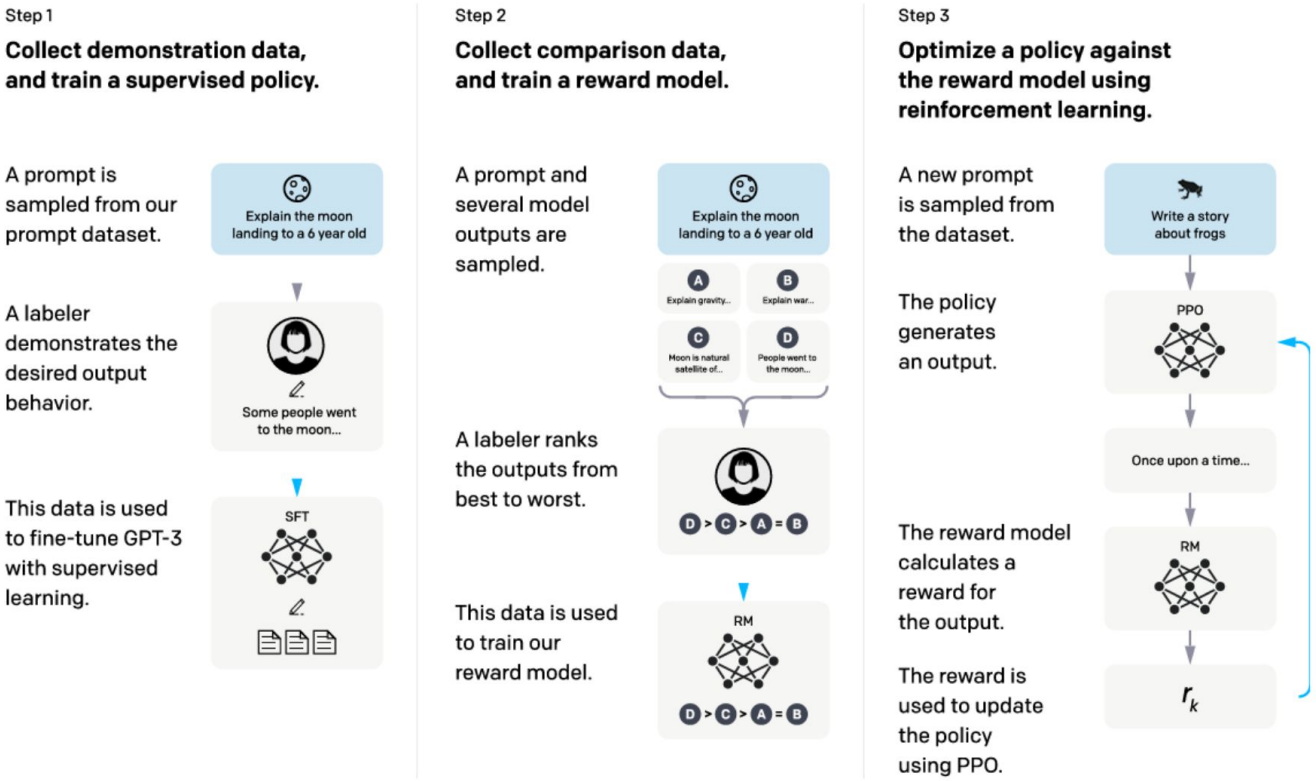
Есть ли память у ChatGPT?



ChatGPT



ChatGPT



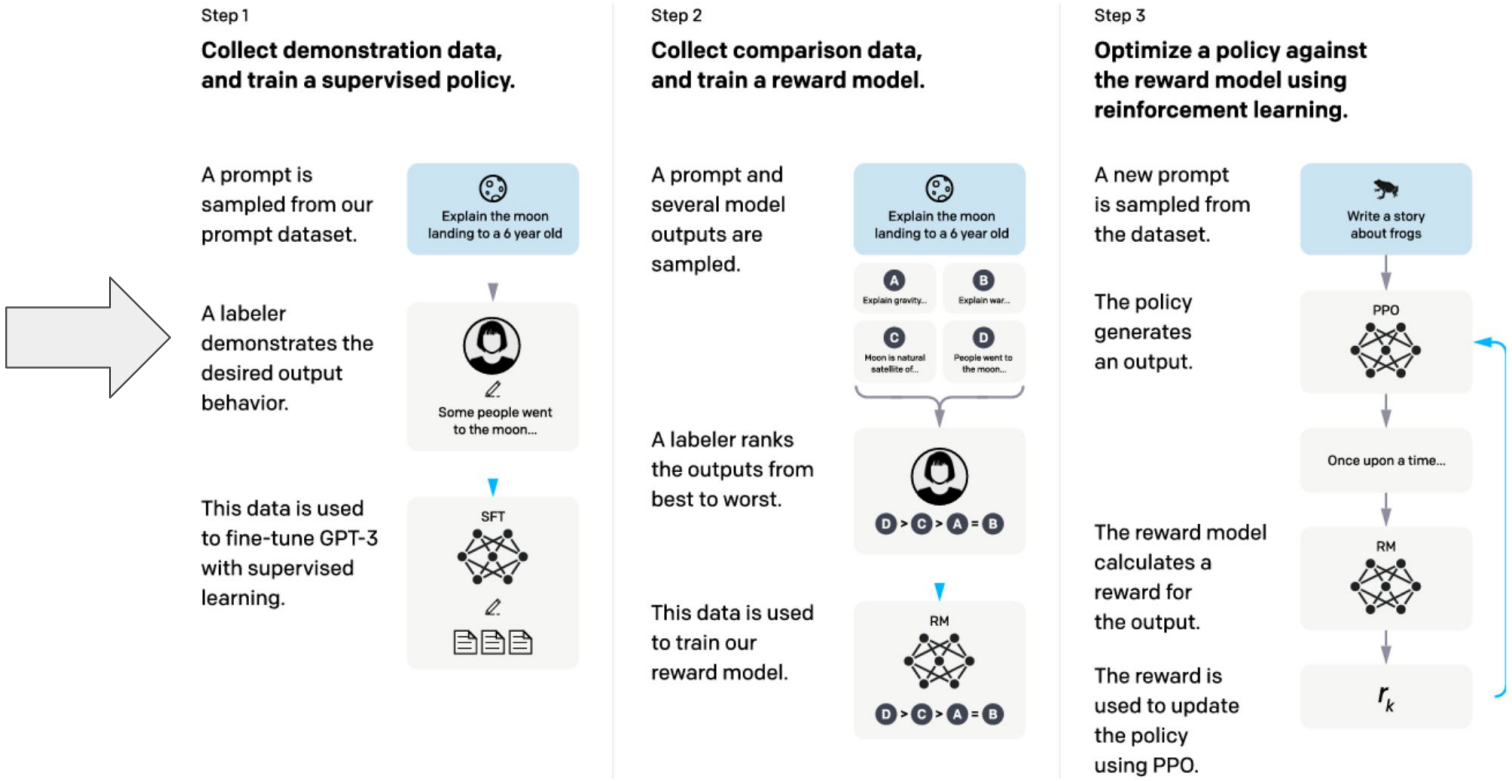
Source: https://arxiv.org/abs/2203.02155

Почему нам не подойдет просто генеративная модель (GPT)?

Запрос: Какие продукты входят в безглютеновую диету?

Ответ: Безглютеновая диета продукты включают рис, кукурузу, картофель, овощи, фрукты, мясо, рыбу, яйца, молочные продукты без добавок, бобовые, орехи. Исключаются пшеница, рожь, ячмень, а также все продукты с мукой из этих злаков.

ChatGPT. Шаг 1: Дообучение на диалогах людей.



Source: https://arxiv.org/abs/2203.02155

Почему мы не остановимся после шага 1?

Запрос: Какие продукты входят в безглютеновую диету?

Ответ: Безглютеновая диета исключает продукты, содержащие глютен (пшеницу, ячмень, рожь). В неё могут входить:

– свежие овощи и фрукты,

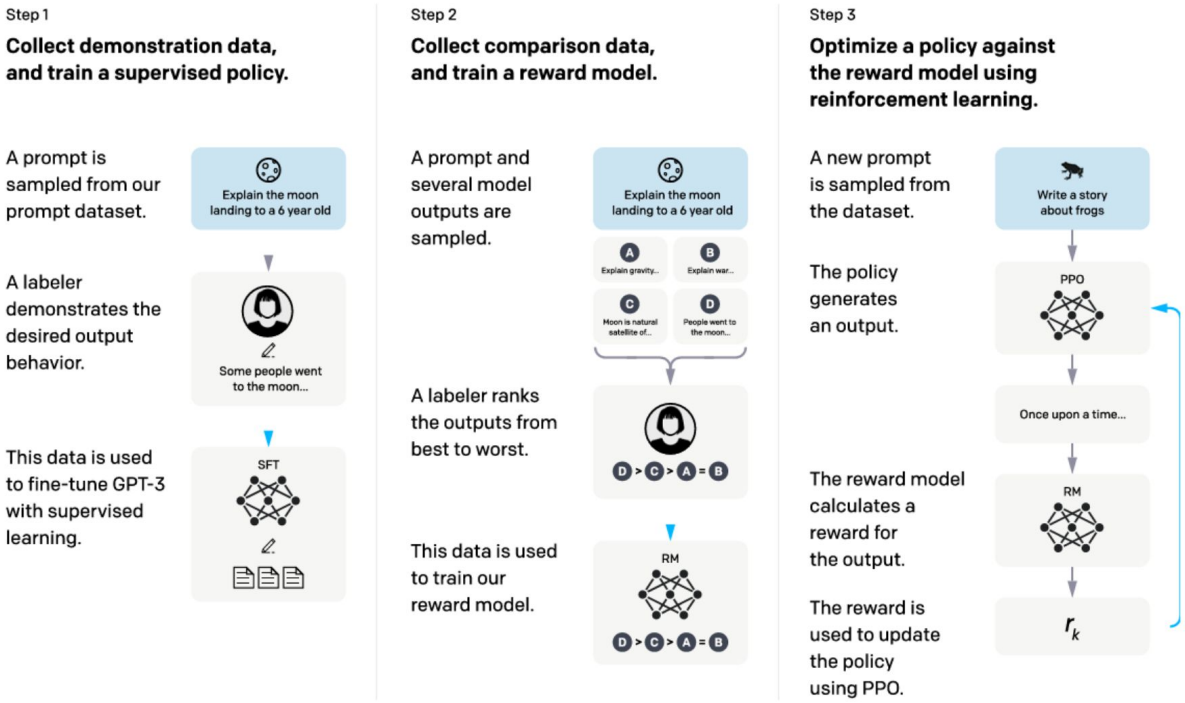
– мясо, рыба и яйца,

– бобовые, рис, гречка, кукуруза,

– безглютеновые макароны и хлеб,

– орехи и семена.

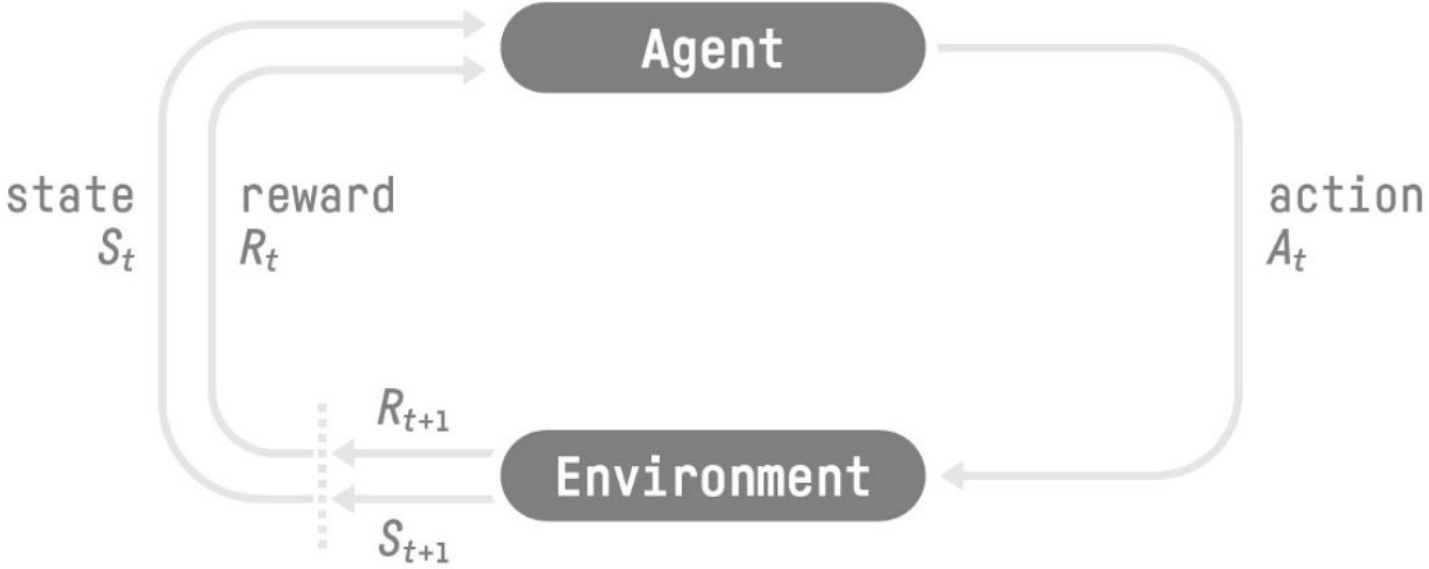
ChatGPT. Шаг 2: Обучение reward-модели



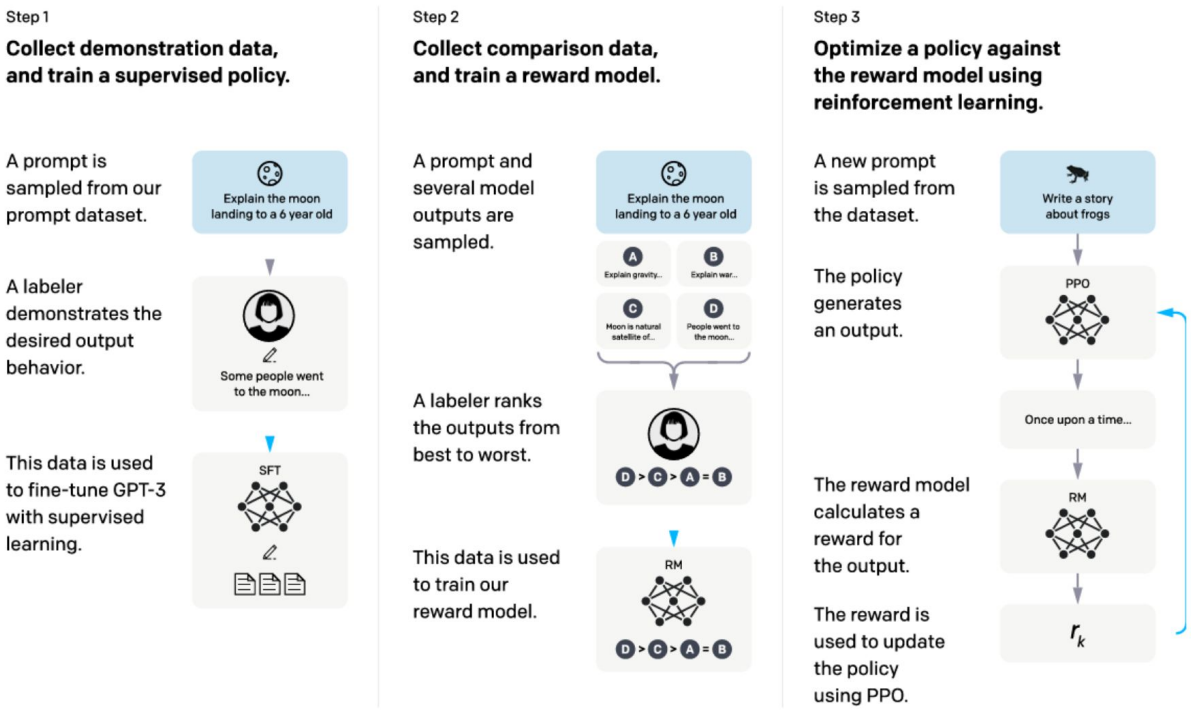
Source: https://arxiv.org/abs/2203.02155

Обучение с подкреплением

Обучение с подкреплением



ChatGPT. Шаг 3: Дообучение GPT с использованием reward-модели



Source: https://arxiv.org/abs/2203.02155

Почему мы не остановимся после шага 2?

Запрос: Какие продукты входят в безглютеновую диету?

Ответ: В безглютеновую диету входят продукты, которые **не содержат глютен**, белок, присутствующий в пшенице, ячмене и ржи. Вот основные безопасные категории:

– **Овощи и фрукты** (свежие, замороженные без добавок)
– **Мясо, птица, рыба** (без панировки)
– **Яйца, молочные продукты**
– **Крупы**: рис, гречка, киноа, кукуруза, просо
– **Бобовые**: фасоль, чечевица, нут
– **Безглютеновая выпечка**: хлеб из рисовой, кукурузной или миндальной муки
– **Орехи и масла**, если не обработаны

📝 *Важно*: всегда проверяйте состав на упаковке — глютен может скрываться в соусах, приправе, колбасе и даже в мороженом.

Шаг 2: Обучение reward-модели

1. **Сбор данных предпочтений:**
   - Люди получают запрос (prompt) и 2–4 разных ответа.
   - Они упорядочивают ответы от лучшего к худшему (или выбирают лучший).
2. **Формирование пар сравнения:**
   - Создаются пары (A, B), где A предпочтительнее B.
3. **Подготовка обучающего датасета:**
   - Каждой паре сопоставляется метка: "A лучше B".
4. **Построение архитектуры RM:**
   - Модель принимает `(prompt, response)` и выдаёт скалярное значение ***r*** — **reward score**.
5. **Обучение (минимизация функции потерь)**

Функция потерь reward-модели

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} \left[\log \left(\sigma \left(r_\theta (x, y_w) - r_\theta (x, y_l) \right) \right) \right]$$

Шаг 3: RLHF с PPO

Дообучаем исходную GPT-модель:

```
             +-------------------------+
prompt -->   |     Transformer (GPT)   | --> logits (для выбора токенов)
             |                         | --> V(s)  (ценностная функция)
             +-------------------------+
```

- logits - прогноз следующего токена (как и раньше)
- V(s) - ценностная функция (дополнительная голов)

V(s) - ценностная функция

- Ценностная функция V(s) - это оценка “ценности” состояния (сколько награды в среднем по всем генерациям из данного промпта мы можем получить, находясь в данном состоянии)?
- Мы хотим, чтобы V(s) приближала прогноз reward-модели, то есть в среднем V(s) примерно была равным R каждого промпта

$$A \text{ (Advantage)} = V(s) - R$$

Value-loss

- Ценностная функция V(s) - это оценка “ценности” состояния (сколько награды в среднем по всем генерациям из данного промпта мы можем получить, находясь в данном состоянии)?
- Мы хотим, чтобы V(s) приближала прогноз reward-модели, то есть в среднем V(s) примерно была равным R каждого промпта

Поэтому для головы, прогнозирующей V(s), лосс у нас:

$$L^{\text{value}} = \mathbb{E}_t \left[(V_\theta(s_t) - R_t)^2 \right]$$

Policy-loss

- Как мы будем обновлять саму стратегию (генерацию токенов GPT)?

$$L = -\min(r \times A, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon) \times A)$$

Policy-loss: r и A

1. Модель сейчас выбирает слово a в контексте s с вероятностью $\pi_\theta(a|s)$.
2. У нас есть **старая версия модели** с вероятностью выбора этого же слова $\pi_{\text{old}}(a|s)$.
3. Мы считаем отношение вероятностей:

$$r = \frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$$

Это показывает, насколько модель изменила свои предпочтения для слова a.

4. Если $r > 1$, значит модель стала выбирать слово a чаще, если $r < 1$ — реже.

Теперь про advantage A:

- Это число показывает, насколько хорош выбор слова a в контексте s.
- Если A положительное — слово хорошее, модель должна выбрать его чаще.
- Если A отрицательное — слово плохое, нужно выбирать реже.

Policy-loss: clipping

Что делает policy loss?

- Он хочет **увеличить вероятность хороших слов (где $A > 0$)** и **уменьшить вероятность плохих (где $A < 0$)**.
- Но **с одной оговоркой**: мы не хотим, чтобы вероятность слова изменилась слишком резко.

Для этого есть clipping (ограничение):

- Если отношение r ушло далеко от 1 (слишком большое или слишком маленькое), мы **сжимаем** (clip) его в диапазон $[1 - \epsilon, 1 + \epsilon]$.
- Так модель не сделает слишком большой скачок — обновления будут плавными и стабильными.

Policy-loss

- Как мы будем обновлять саму стратегию (генерацию токенов GPT)?

$$L = -\min(\underline{r \times A}, \mathrm{clip}(r, 1-\epsilon, 1+\epsilon) \times A)$$

Хотим максимизировать вероятность действий с положительным A (константа), то есть при обучении будем обновлять веса так, чтобы максимизировать r при A>0.

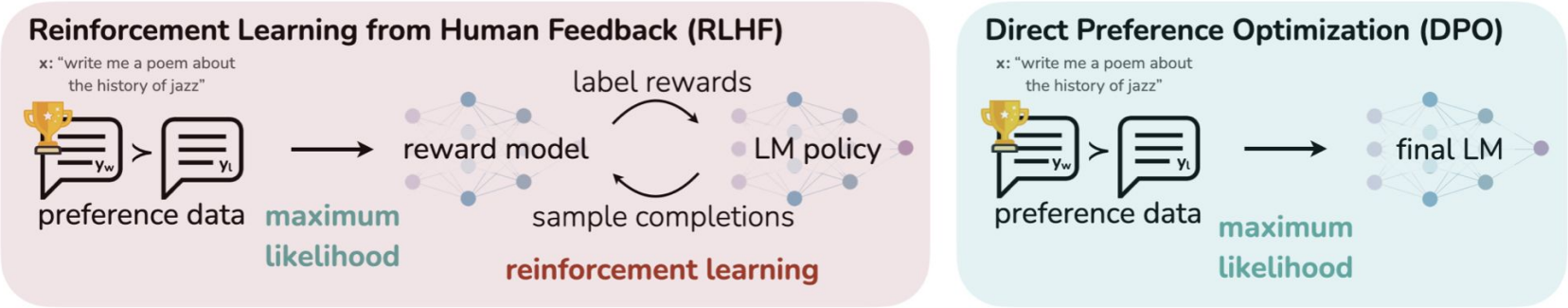
Итоговый лосс

Сумма:

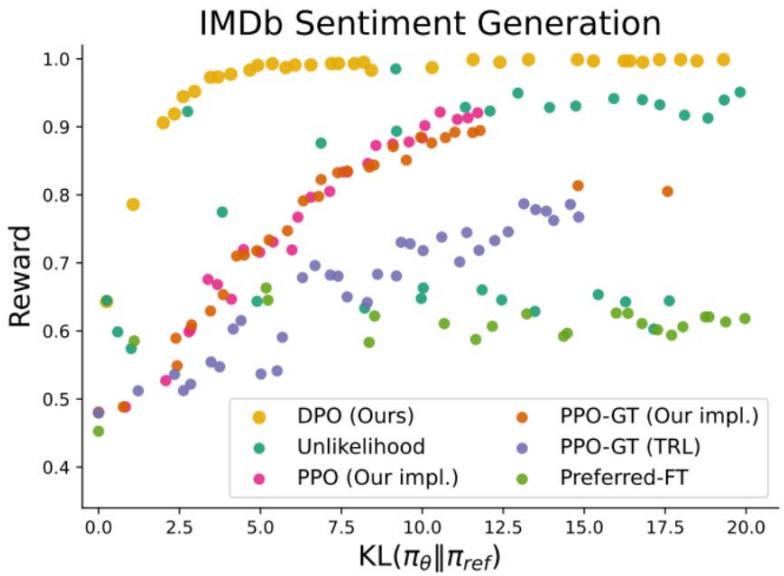
- Value loss
- Policy loss
- Entropy loss (для разнообразных ответов)

DPO - Direct Policy Optimization

Вместо того чтобы строить reward-модель и проводить сложное RL-обучение, DPO формулирует задачу как **обучение модели классифицировать, какой из двух ответов лучше** и подстраиваться под это.



DPO vs PPO



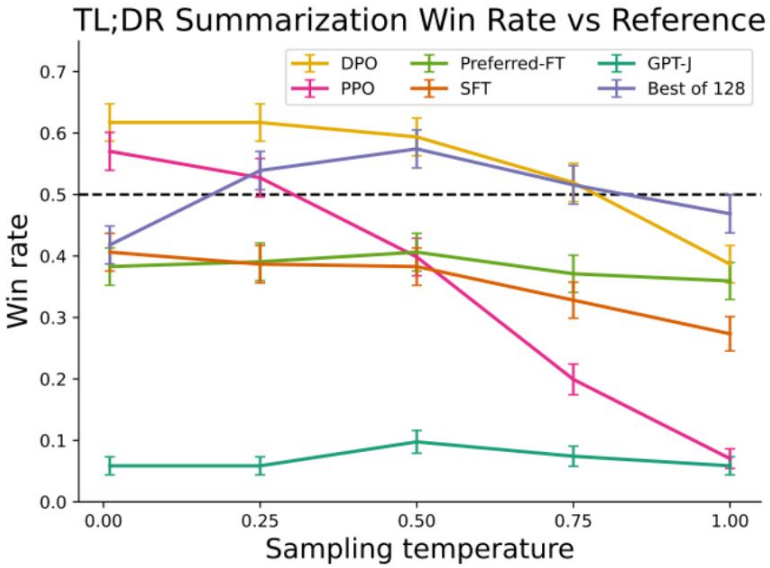
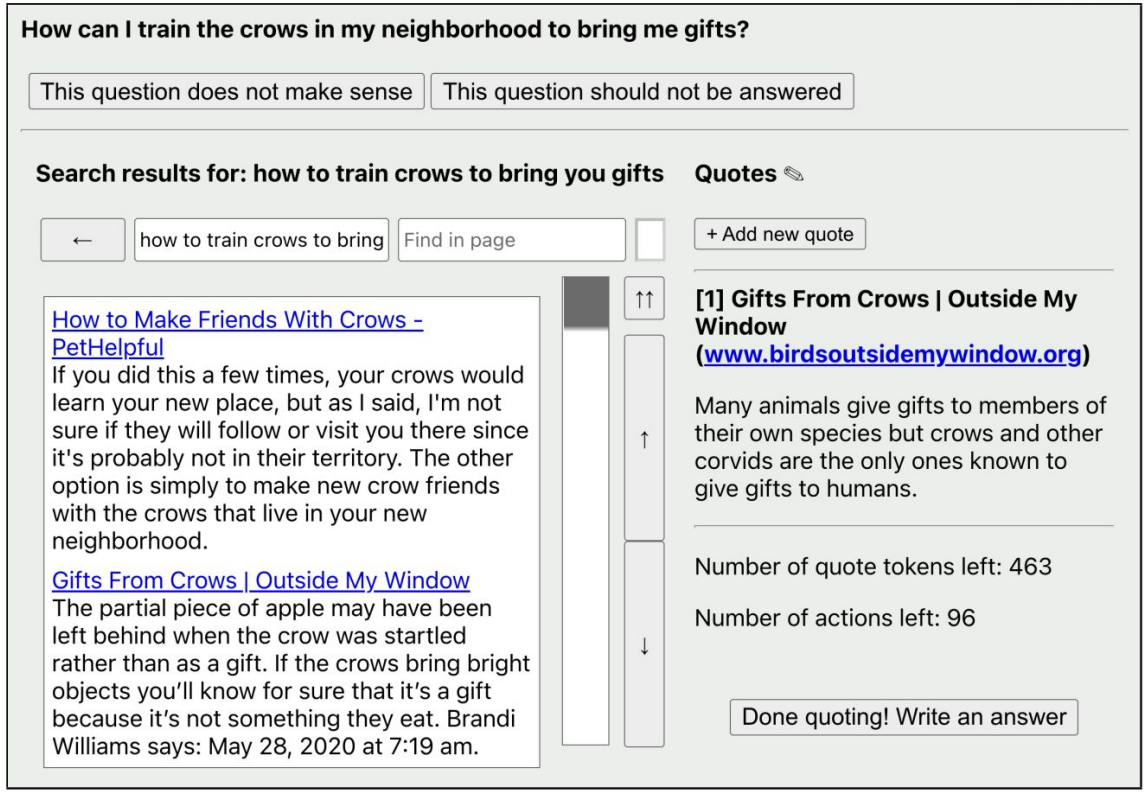


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO's best-case performance on summarization, while being more robust to changes in the sampling temperature.

WebGPT


How can I train the crows in my neighborhood to bring me gifts?
This question does not make sense
This question should not be answered
Search results for: how to train crows to bring you gifts
Quotes
←
how to train crows to bring
Find in page
+ Add new quote
How to Make Friends With Crows - PetHelpful
If you did this a few times, your crows would learn your new place, but as I said, I'm not sure if they will follow or visit you there since it's probably not in their territory. The other option is simply to make new crow friends with the crows that live in your new neighborhood.
Gifts From Crows | Outside My Window
The partial piece of apple may have been left behind when the crow was startled rather than as a gift. If the crows bring bright objects you'll know for sure that it's a gift because it's not something they eat. Brandi Williams says: May 28, 2020 at 7:19 am.
↑↑
↑
↓
[1] Gifts From Crows | Outside My Window (www.birdsoutsidemywindow.org)
Many animals give gifts to members of their own species but crows and other corvids are the only ones known to give gifts to humans.
Number of quote tokens left: 463
Number of actions left: 96
Done quoting! Write an answer

WebGPT

Table 1: Actions the model can take. If a model generates any other text, it is considered to be an invalid action. Invalid actions still count towards the maximum, but are otherwise ignored.

Command	Effect
`Search <query>`	Send `<query>` to the Bing API and display a search results page
`Clicked on link <link ID>`	Follow the link with the given ID to a new page
`Find in page: <text>`	Find the next occurrence of `<text>` and scroll to it
`Quote: <text>`	If `<text>` is found in the current page, add it as a reference
`Scrolled down <1, 2, 3>`	Scroll down a number of times
`Scrolled up <1, 2, 3>`	Scroll up a number of times
`Top`	Scroll to the top of the page
`Back`	Go to the previous page
`End: Answer`	End browsing and move to answering phase
`End: <Nonsense, Controversial>`	End browsing and skip answering phase

WebGPT

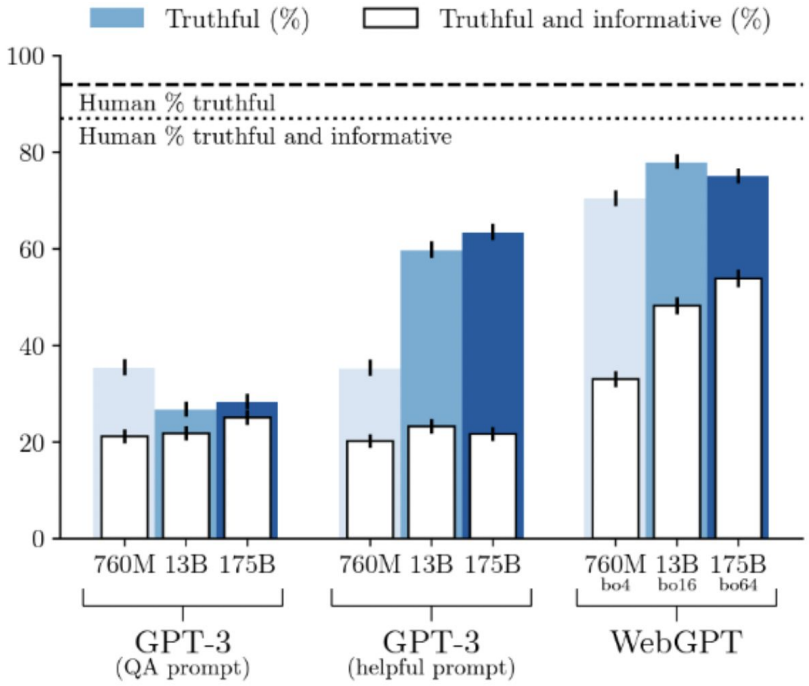


Figure 3: TruthfulQA results. The amount of rejection sampling (the n in best-of-n) was chosen to be compute-efficient (see Figure 8). Error bars represent ± 1 standard error.

RAG

Галлюцинации LLM

How many 'm's are in the word 'Weather'?

There is one 'm' in the word 'Weather'.

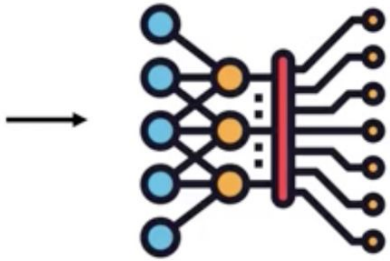
Are you sure?

Apologies for the oversight. You're right; there are no 'm's in the word 'Weather'. Thank you for pointing it out.

RAG

В случае, если галлюцинирование критично и/или необходимо учесть новые данные, удобно воспользоваться внешними базами данных

Вопрос клиента: «Каковы условия досрочного погашения потребительского кредита?»

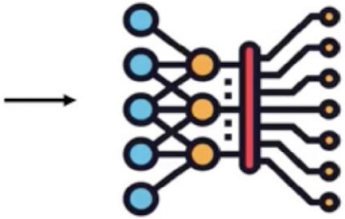


RAG

В случае, если галлюцинирование критично и/или необходимо учесть новые данные, удобно воспользоваться внешними базами данных

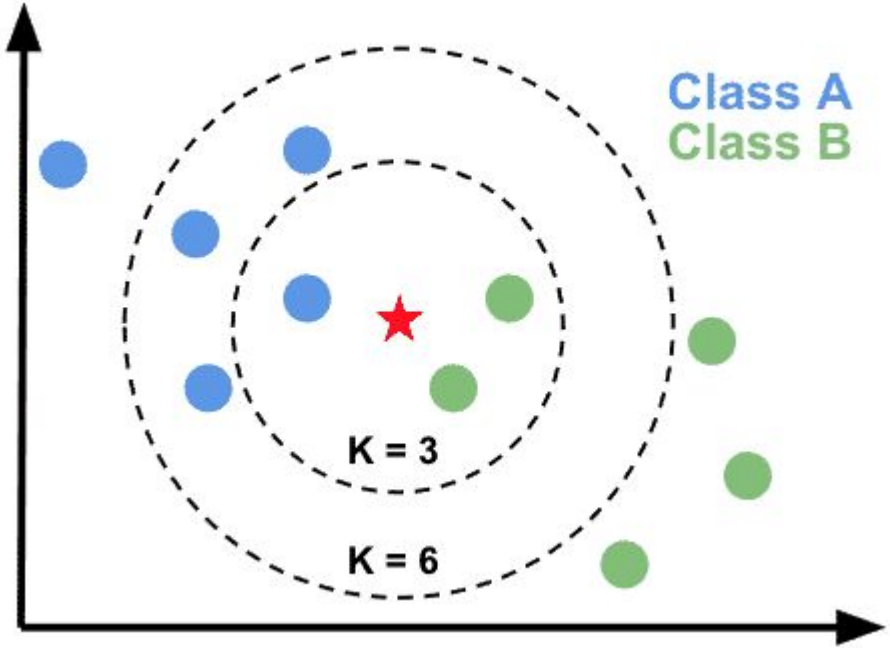
Вопрос клиента: «Каковы условия досрочного погашения потребительского кредита?»

Контекст: *найденные релевантные факты из базы данных*

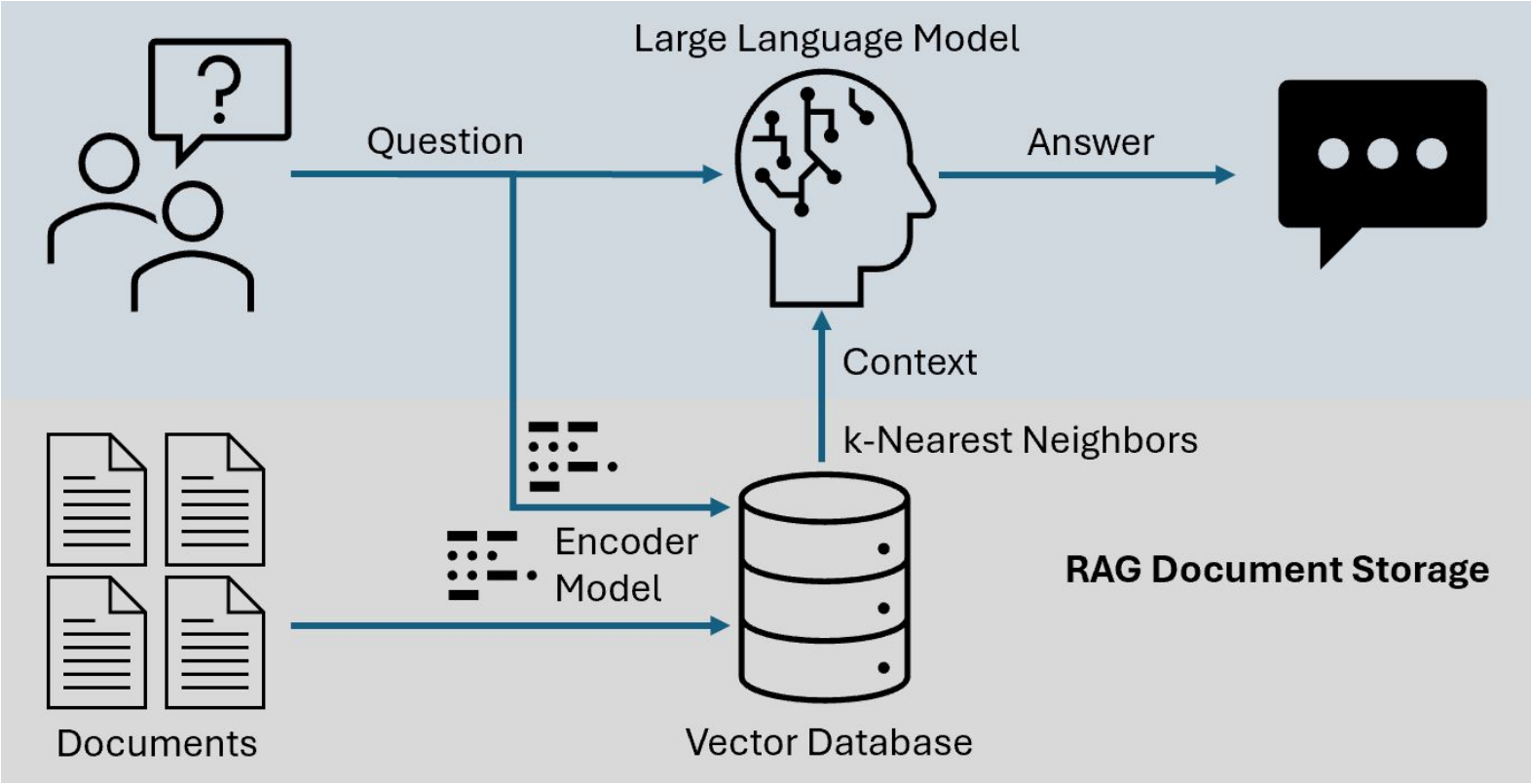


Данные из векторной базы (найдены в PDF с тарифами банка):
«Досрочное погашение возможно после 6 месяцев выплат. Комиссия – 1% от суммы погашения. Требуется письменное уведомление за 3 рабочих дня.»

KNN



RAG (Retrieval Augmented Generation)



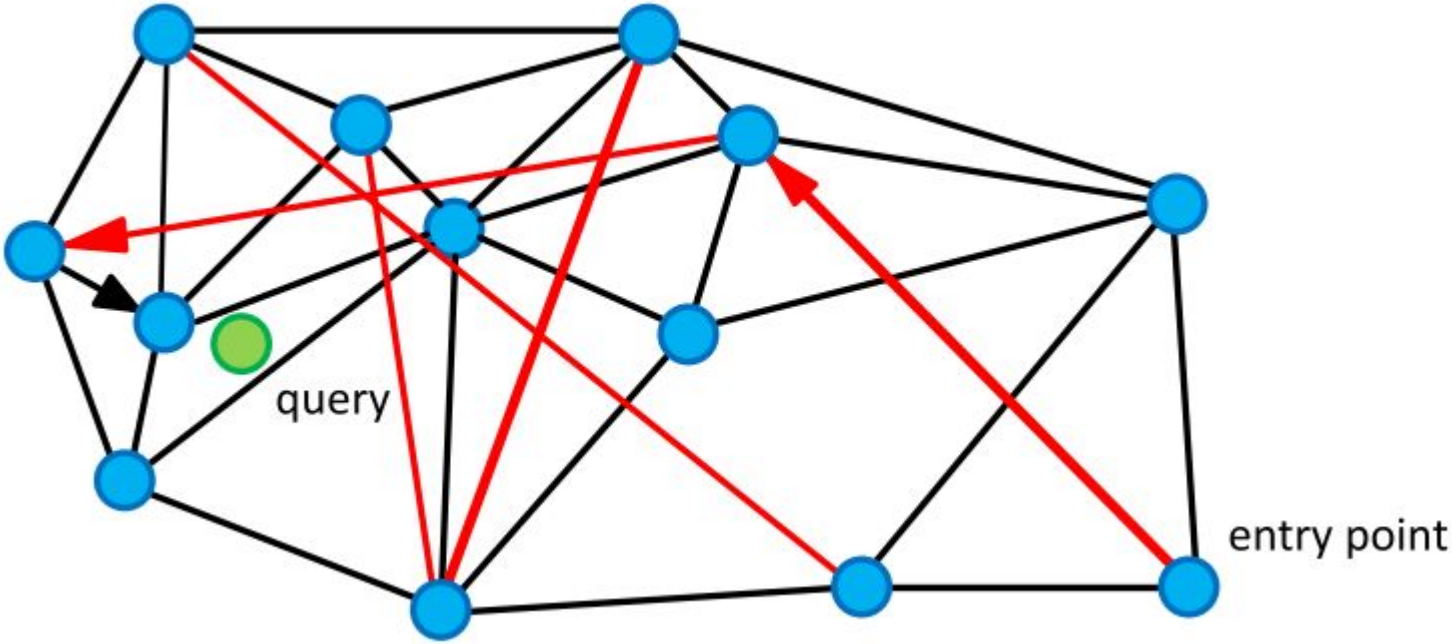
Далее в программе

1. Что надо знать?
2. Методы поиска эмбеддингов: Approximate NN (HNSW, FAISS)
3. Стратегии RAG
4. Практика (LangChain)
5. Агенты

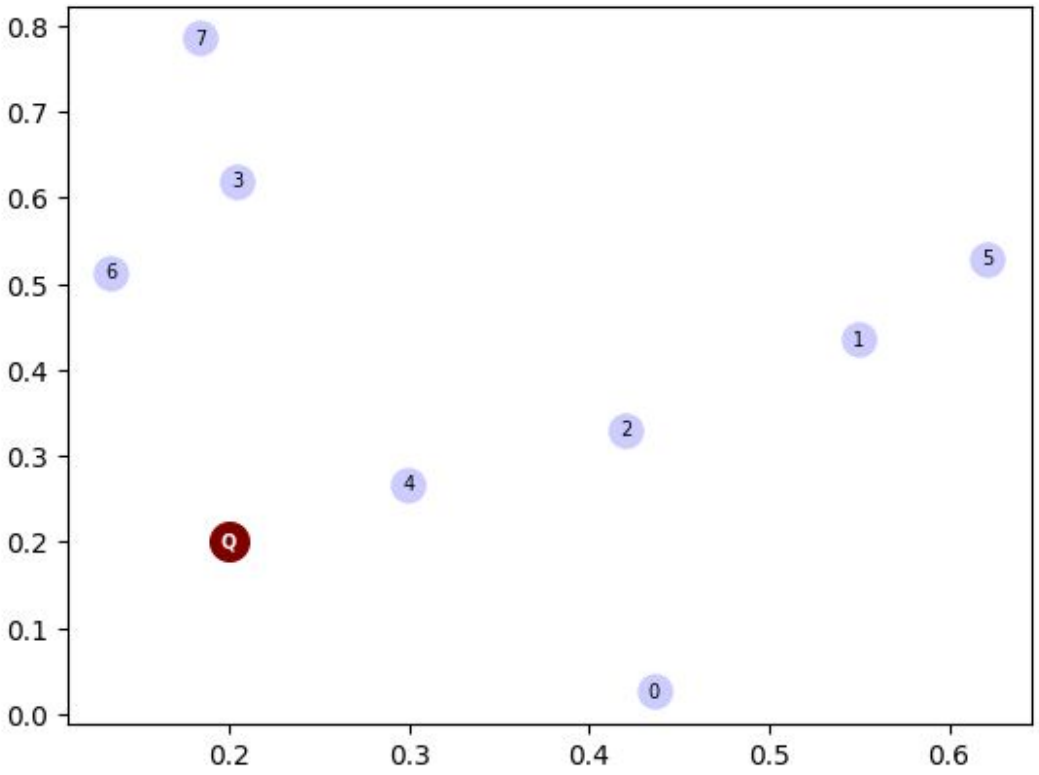
Инструменты для RAG

Процесс	Описание	Примеры библиотек / сервисов	Примечания
1. Загрузка и хранение данных	Импорт, хранение и управление исходными текстами, документами и метаданными	BigQuery, Pinecone, Weaviate, Milvus, SQL/NoSQL БД	Обеспечивают быстрый доступ и масштабируемость
2. Разбиение на чанки (chunking)	Деление больших текстов на более мелкие части для обработки и векторизации	LangChain (TextSplitter), Haystack, собственные скрипты	Важно для контроля длины контекста и качества эмбеддингов
3. Генерация эмбеддингов	Преобразование текстовых чанков в векторные представления	Sentence-BERT, OpenAI Embeddings, Cohere, HuggingFace Transformers	Может быть как облачный API, так и локальные модели
4. Индексация векторов и поиск	Создание индексов для эффективного поиска ближайших соседей по эмбеддингам	FAISS, Annoy, HNSWlib, ScaNN, Pinecone, Milvus	Позволяет быстро находить релевантные документы
5. Оркестрация и интеграция	Управление процессом поиска, генерации ответов, объединение результатов	LangChain, Haystack, LlamaIndex	Координируют взаимодействие между эмбеддингами, поиском и LLM
6. Генерация ответа LLM	Использование больших языковых моделей для генерации текста на основе найденных документов	OpenAI GPT, Anthropic Claude, HuggingFace LLMs	Работает с контекстом, сформированным из результатов поиска

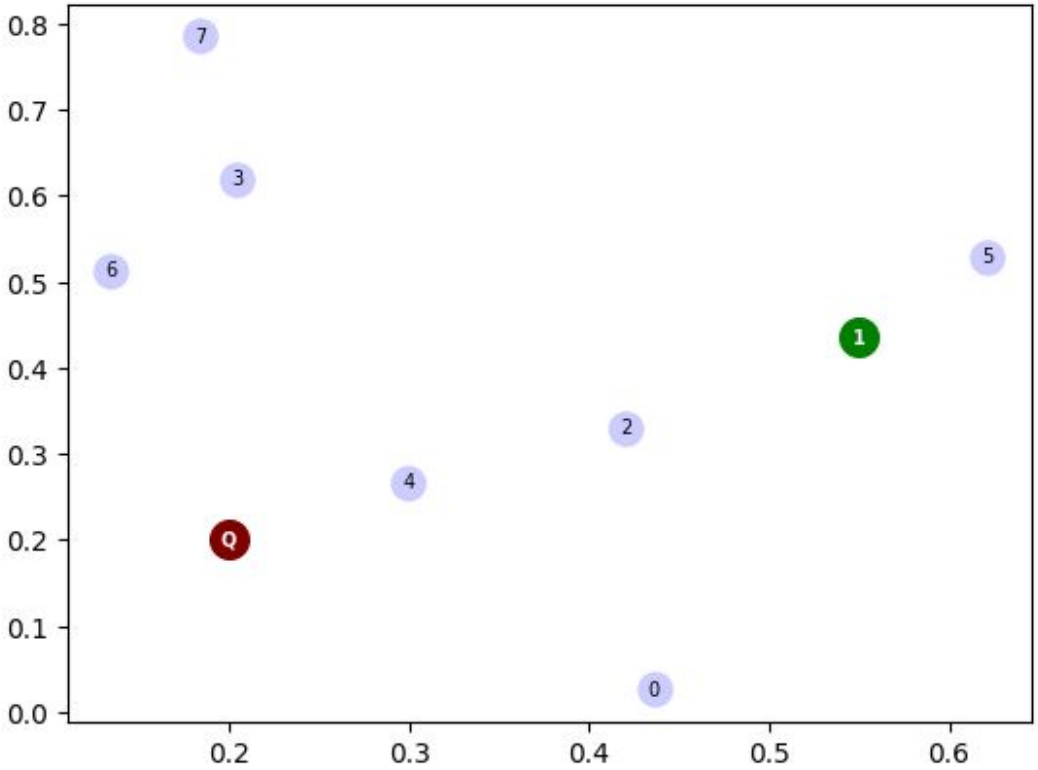
Approximate NN: NSW -> HNSW



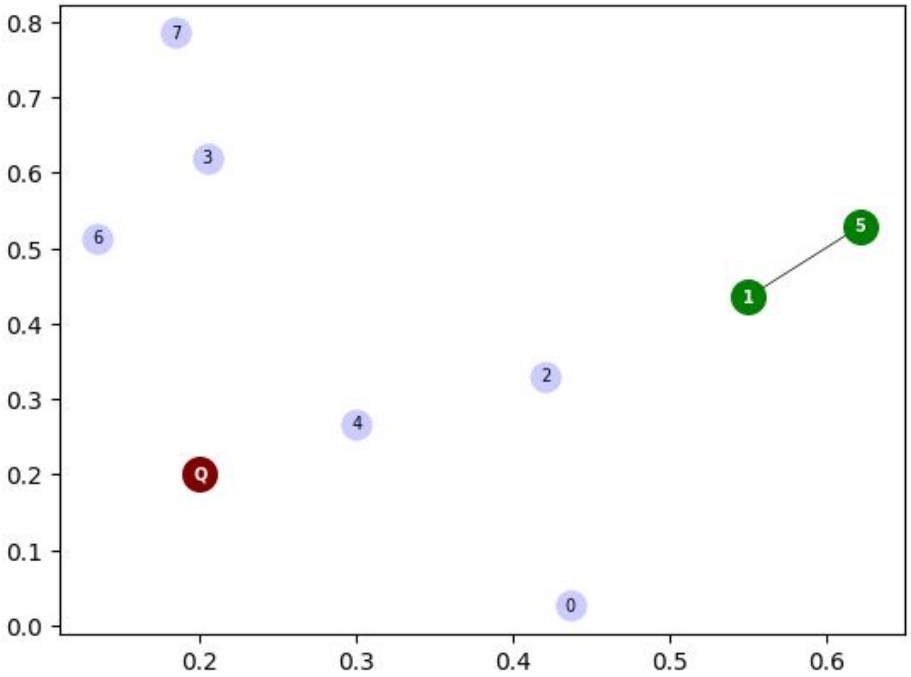
Navigable Small World (NSW): строим граф



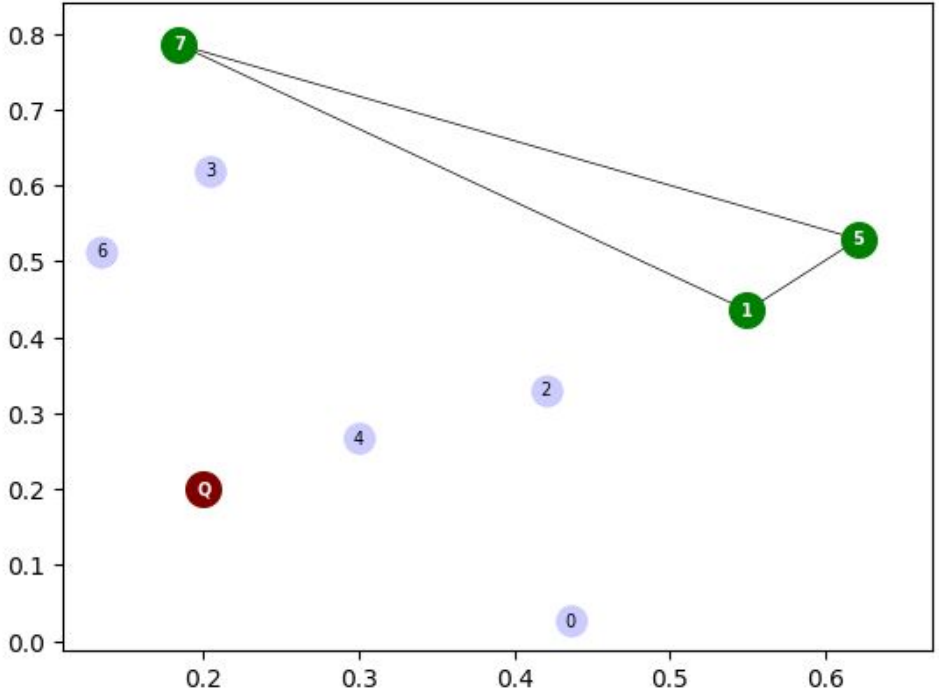
Берем случайную вершину (эмбеддинг) и добавляем в граф



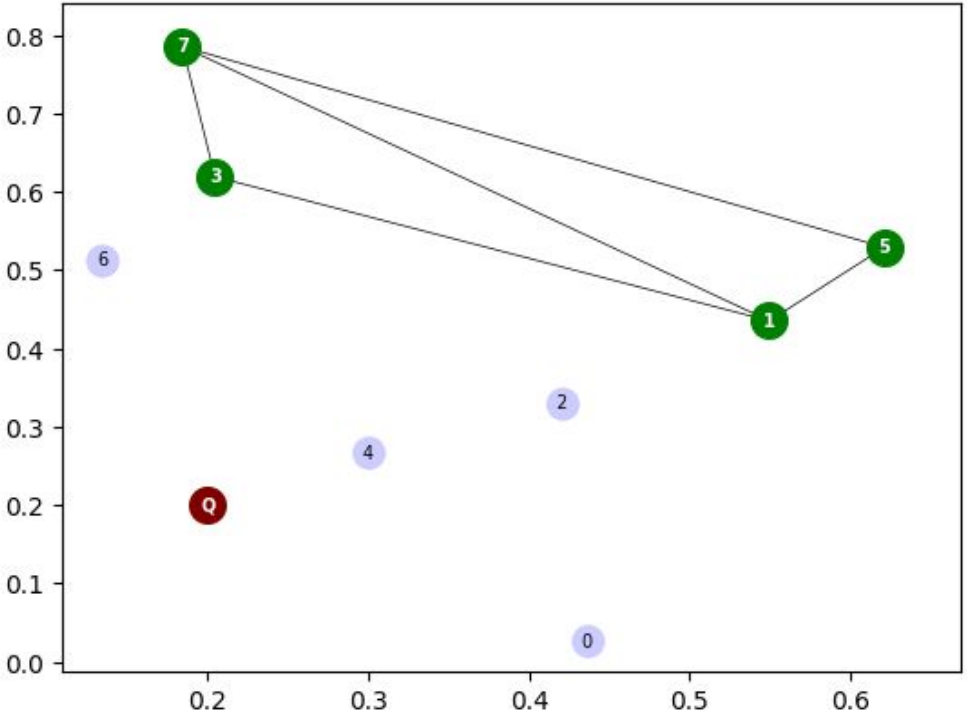
Каждую следующую случайную вершину соединяем с двумя ближайшими, которые уже есть в графе



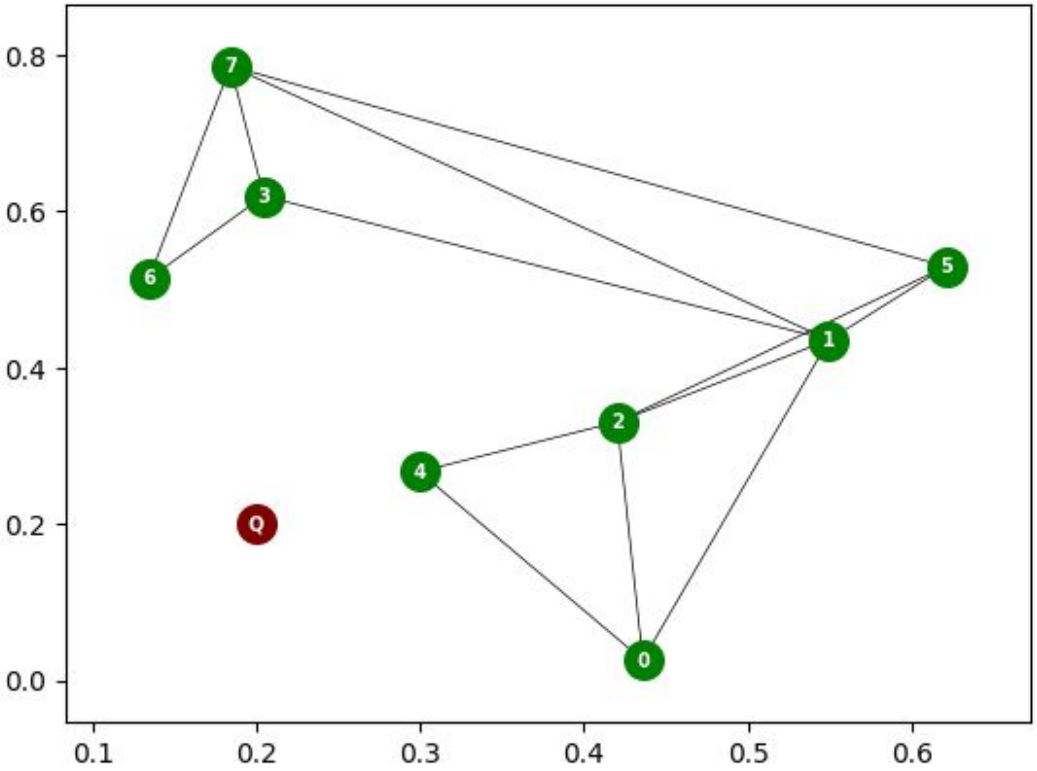
Каждую следующую случайную вершину соединяем с двумя ближайшими, которые уже есть в графе



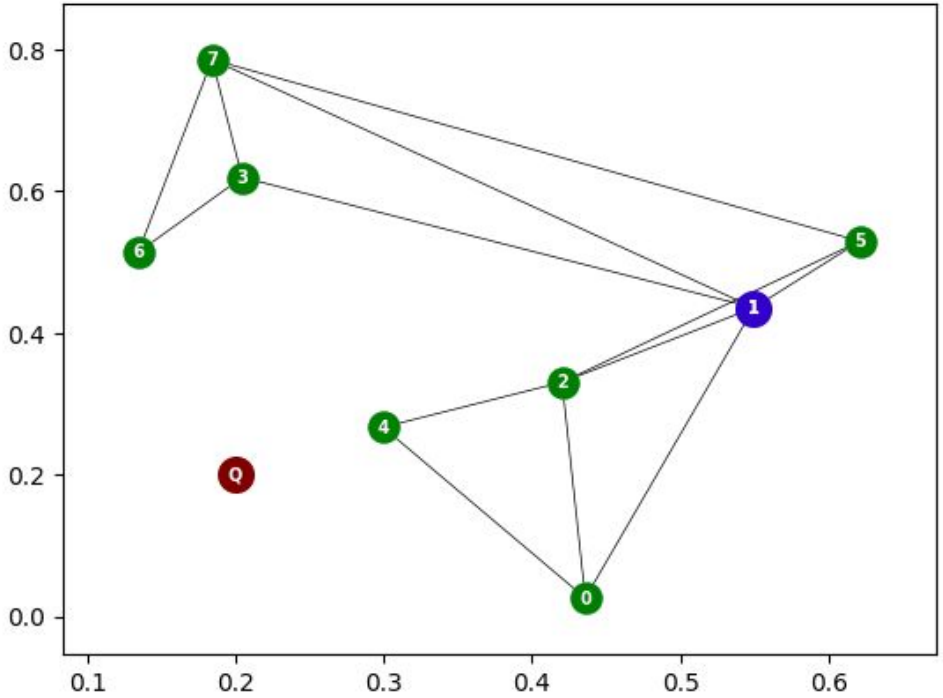
Каждую следующую случайную вершину соединяем с двумя ближайшими, которые уже есть в графе



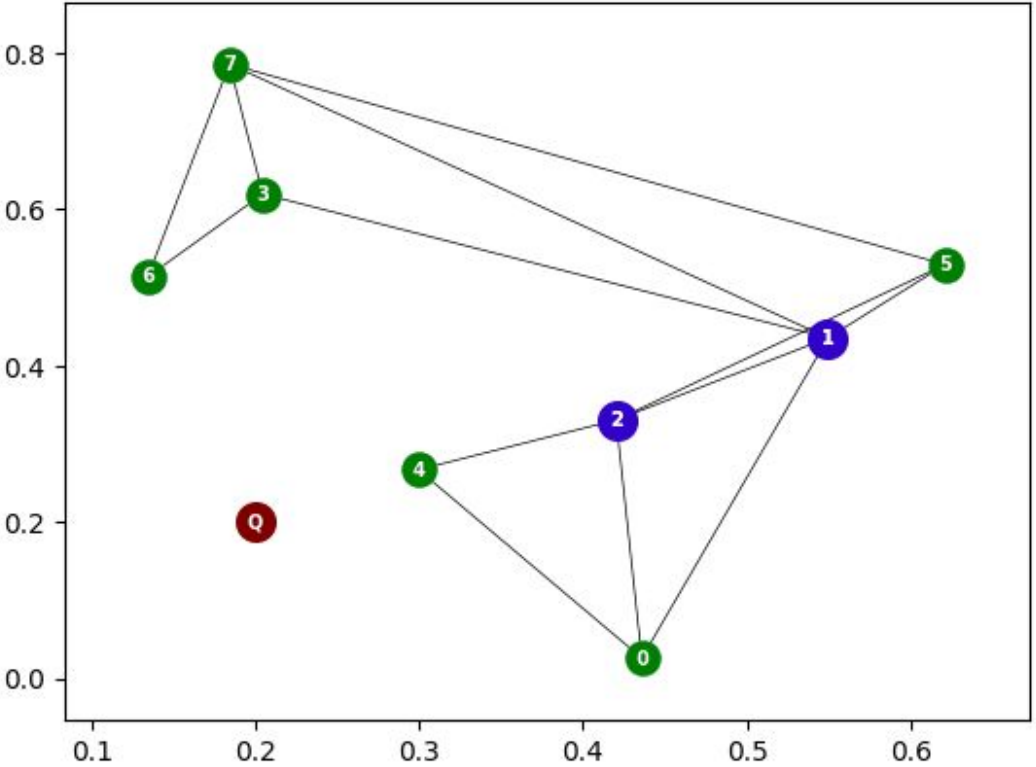
Navigable Small World (NSW): получаем граф



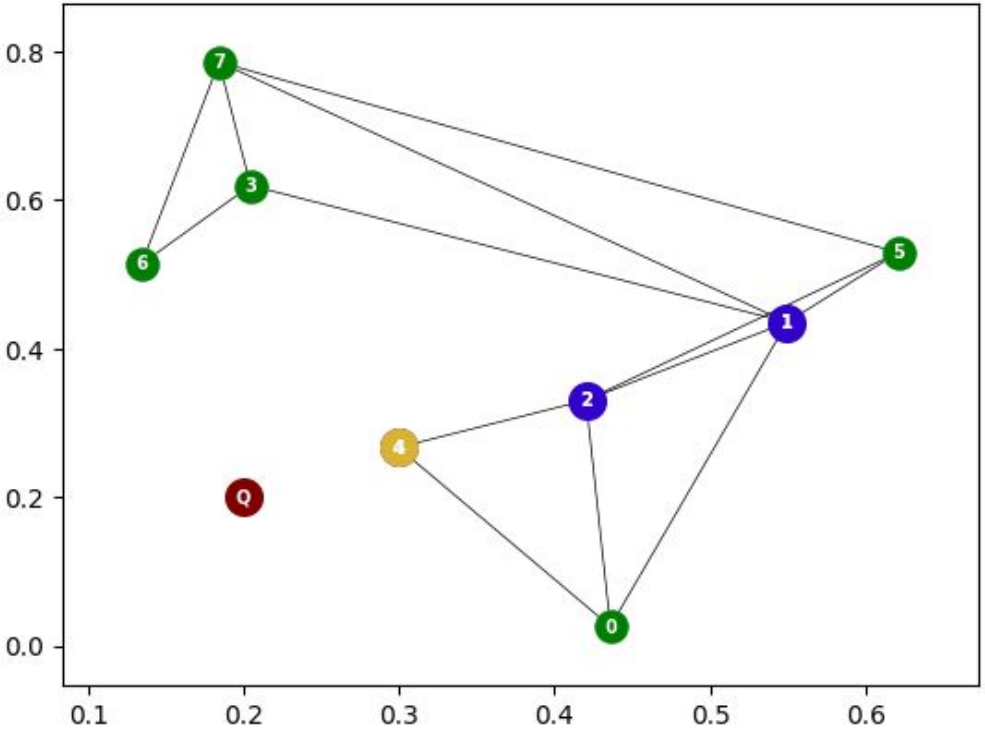
NSW: поиск по графу - берем случайную вершину и считаем расстояние от q до нее и ее соседей



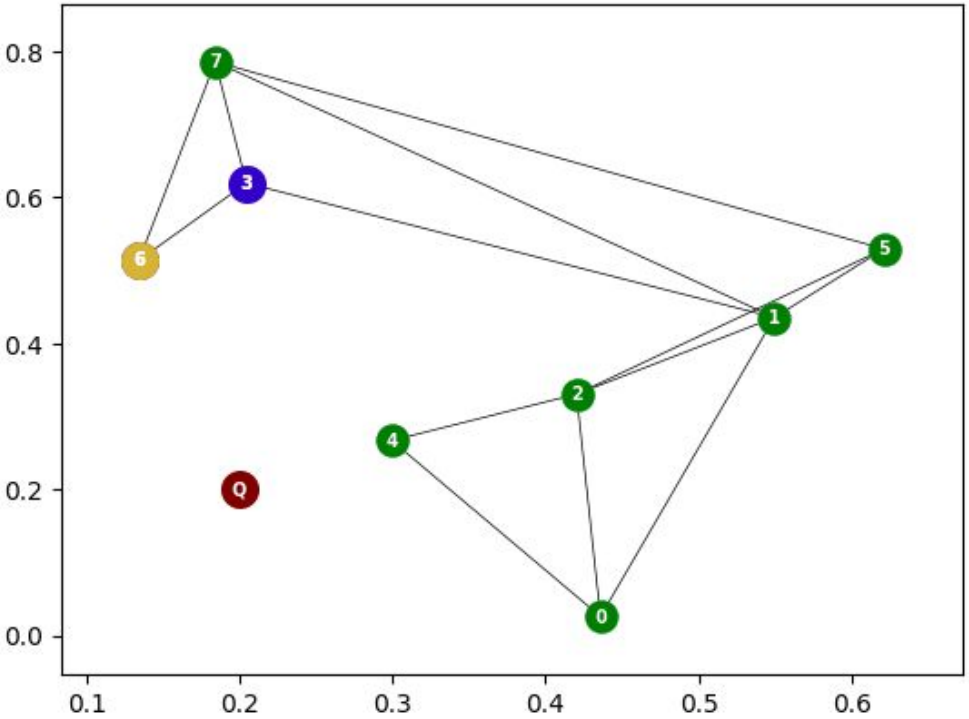
NSW: поиск по графу - идем в ближайшую к q из соседей



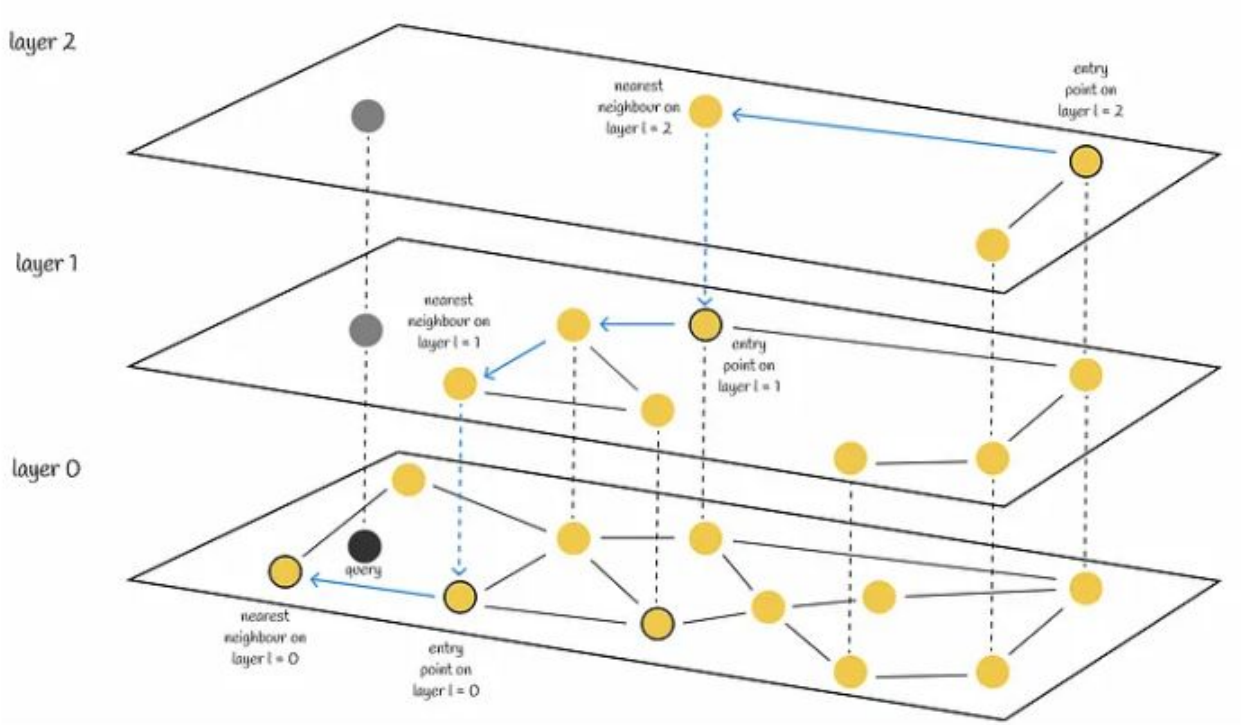
NSW: поиск по графу - идем в ближайшую к q из соседей и из нее повторяем процесс



NSW: итог - получаем приближенного ближайшего соседа. Почему? Зависит от стартовой точки.



HNSW - Hierarchical NSW - когда запросов много!



HNSW - Hierarchical NSW - когда запросов много!

Строим уровневые графы:

- Каждый вектор при добавлении получает случайный максимальный уровень L - то есть точка добавляется на уровни 1, 2, …, L
- На каждом уровне точка соединяется с M ближайшими соседями (по выбранной метрике)

Добавление точки (объекта) в граф:

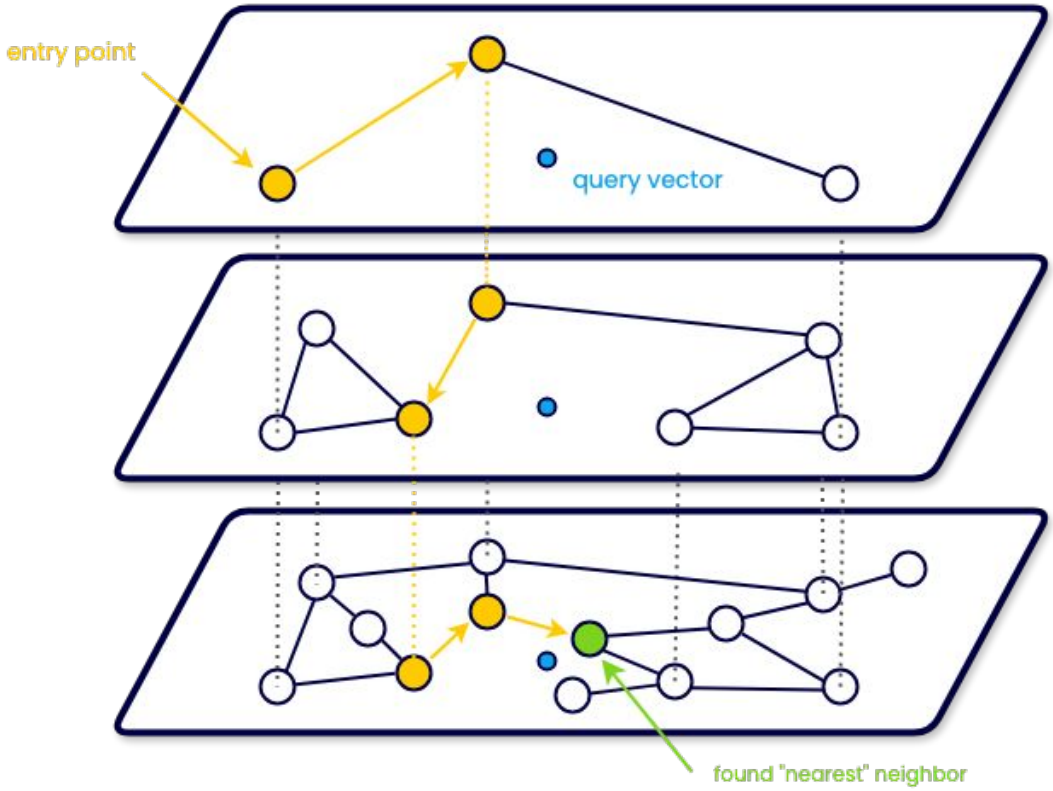
- Задаем точке случайный уровень L
- Добавляем точку на верхний уровень и соединяем с M ближайшими соседями
- Далее добавляем точку на следующий уровень и ищем там M ближайших соседей и так далее

HNSW: поиск по графу

Когда приходит эмбеддинг запроса:

- Начало поиска — стартуем с произвольной (или заранее выбранной) точки на самом верхнем уровне
- Грубая навигация — двигаемся по рёбрам, пока не найдём ближайшую точку на этом уровне
- Спуск вниз — переходим на уровень ниже, используя найденную точку как старт
- Точный поиск — на нижнем (нулевом) уровне ищем ближайших соседей до нужного k.

HNSW: поиск по графу



Как строим эмбеддинги?

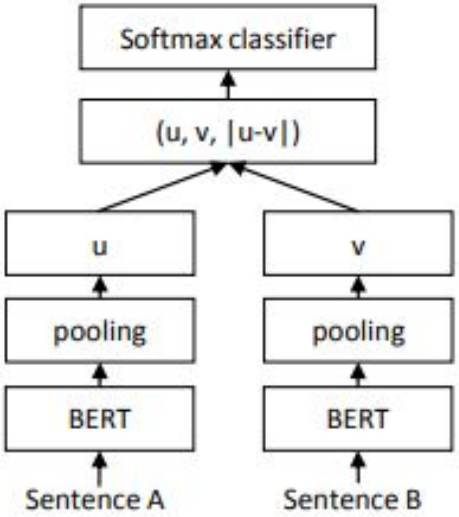


Figure 1: SBERT architecture with classification objective function, e.g., for fine-tuning on SNLI dataset. The two BERT networks have tied weights (siamese network structure).

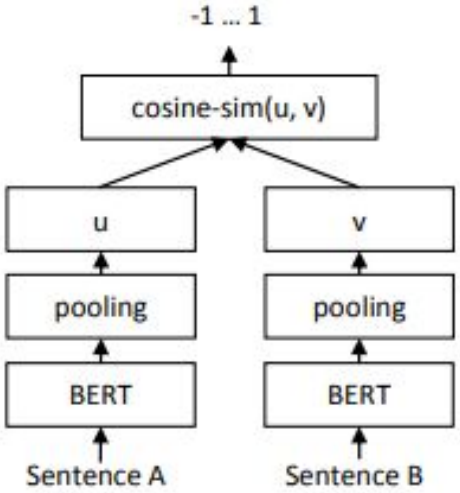


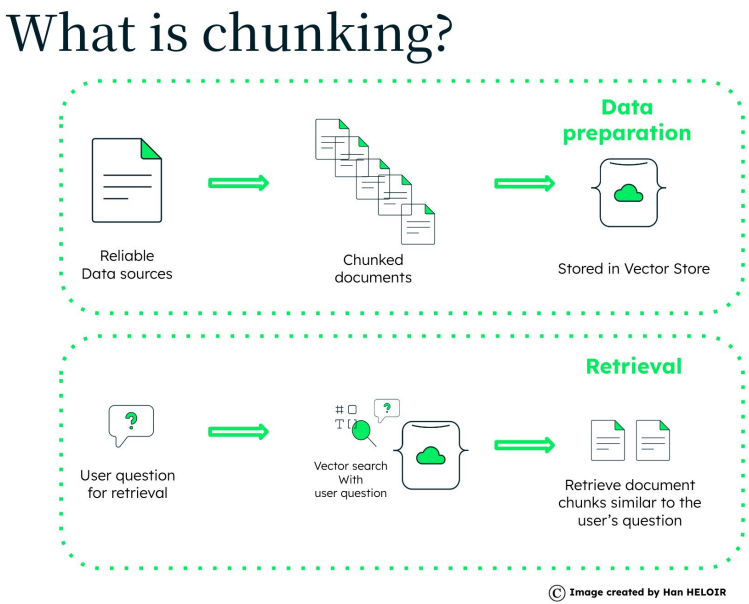
Figure 2: SBERT architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.

Sentence BERT

1. Два одинаковых BERT-энкодера с разделяемыми весами
2. На вход подаются:
    - Пара предложений (или запрос + документ)
3. Сверху на BERT берётся pooling (обычно mean pooling по токенам)
4. Сравниваем полученные эмбеддинги:
    - косинусная близость
    - dot product
5. Обучаем так, чтобы похожие тексты имели близкие векторы, а непохожие — далёкие

Chunking

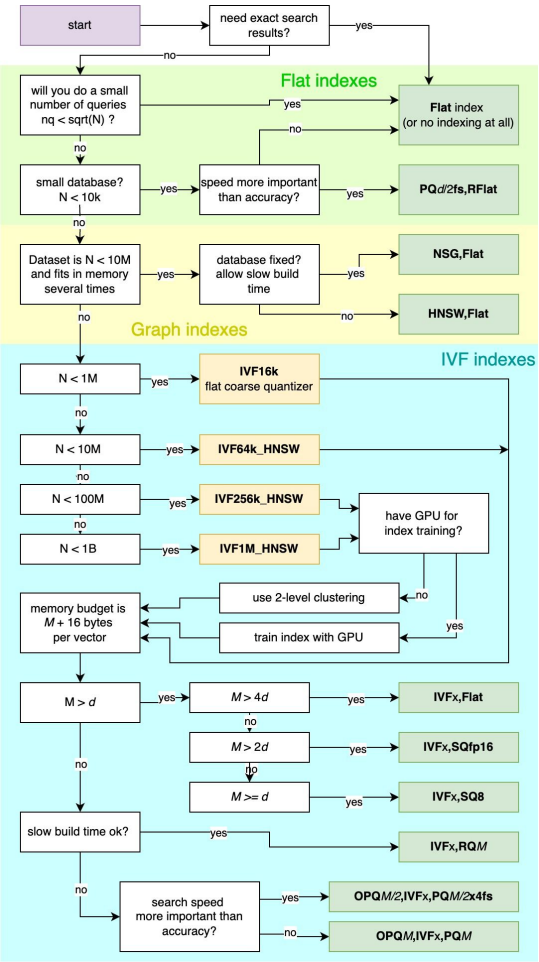
- Разбиваем на куски по N символов или токенов (например, 500–1000 токенов).
- Добавляем перекрытие (overlap) 50–100 токенов, чтобы не терялись переходные фразы (обычно 10-20% от длины чанков)



Библиотеки: FAISS

FAISS (Facebook AI Similarity Search) — это библиотека от Facebook для эффективного поиска ближайших соседей (ANN — Approximate Nearest Neighbors) в больших коллекциях векторов.

FAISS


start
need exact search results?
yes
no
Flat indexes
will you do a small number of queries nq < sqrt(N) ?
yes
Flat index (or no indexing at all)
no
small database? N < 10k
yes
speed more important than accuracy?
no
yes
PQd/2fs,RFlat
no
Dataset is N < 10M and fits in memory several times
yes
database fixed? allow slow build time
yes
NSG,Flat
no
HNSW,Flat
no
Graph indexes
IVF indexes
N < 1M
yes
IVF16k flat coarse quantizer
no
N < 10M
yes
IVF64k_HNSW
no
N < 100M
yes
IVF256k_HNSW
no
N < 1B
yes
IVF1M_HNSW
have GPU for index training?
no
use 2-level clustering
yes
train index with GPU
memory budget is M + 16 bytes per vector
M > d
yes
M > 4d
yes
IVFx,Flat
no
M > 2d
yes
IVFx,SQfp16
no
M >= d
yes
IVFx,SQ8
no
slow build time ok?
yes
IVFx,RQM
no
search speed more important than accuracy?
yes
OPQM/2,IVFx,PQM/2x4fs
no
OPQM,IVFx,PQM

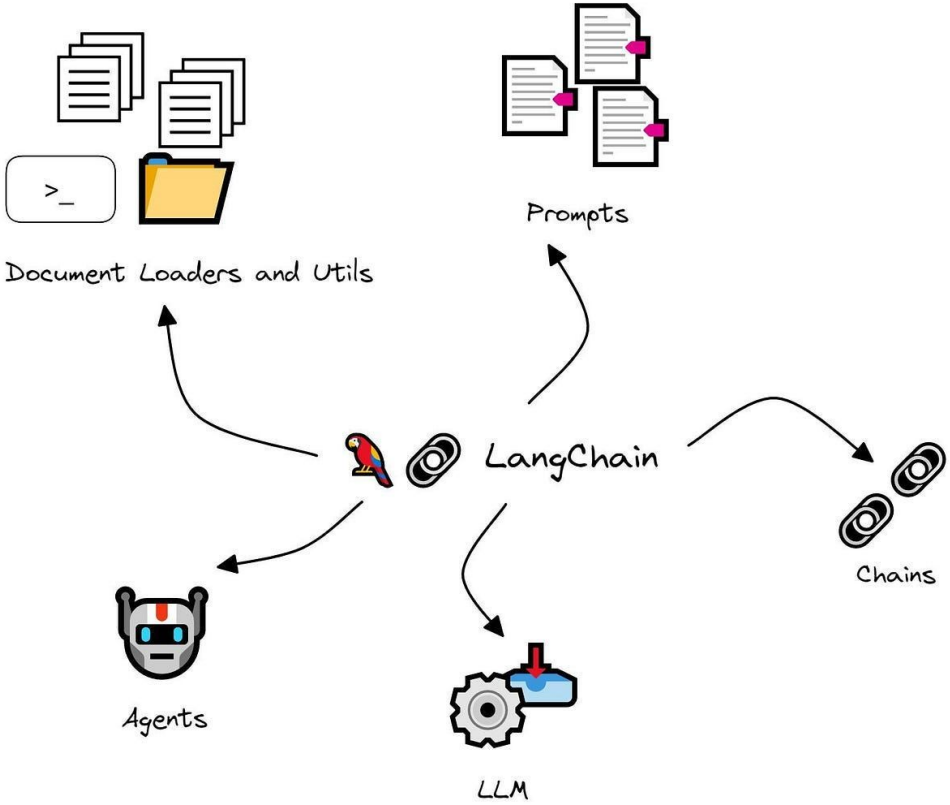
Библиотеки: ChromaDB

ChromaDB — это *векторная база данных*, оптимизированная для хранения и поиска векторных эмбеддингов (часто используется в задачах ИИ, например для semantic search или RAG)

Как это работает:

- Инициализируем ChromaDB — создаем подключение и коллекцию, куда будут складываться документы.
- Настраиваем генерацию эмбеддингов — выбираем модель (например, OpenAI embeddings) и привязывем её к коллекции
- Загружаем документы — добавляем тексты в ChromaDB вместе с уникальными ID и сгенерированными векторами
- Выполняем поиск — по запросу пользователя ищем в ChromaDB те документы, что наиболее близки по смыслу

Библиотеки: LangChain



Практика: создание RAG-системы

https://colab.research.google.com/drive/1Qf6fmhf2dKDt6yutOM5tdcwWlOBk5H8E?usp=sharing